PI Control Design by Pole Placement

1.0 Learning Outcomes....1
2.0 Introduction....1
3.0 The Model and the Controller....2
4.0 Specifying the Closed-Loop Poles....3
5.0 Pole Placement Design....4
6.0 Example: Pole-Placement PI Design for a DC Motor (Simulation) 6
7.0 Summary....8
References....8
Recommended Citation....8

1.0 Learning Outcomes

After studying this ACE-Lab learning guide, you should be able to demonstrate the Learning Outcomes listed in Table 1. On completion of the ACE-Lab learning guide, revisit Table 1 and confirm whether each Learning Outcome has been achieved.

Table 1: *Learning Outcomes from the PI Control Design by Pole Placement Learning Guide*

ID	Learning Outcome	Achieved
1.	Write the closed-loop characteristic equation of a discrete-time PI controller and a first-order model.	☐
2.	Convert a required damping ratio and natural frequency into desired closed-loop poles on the z-plane.	☐
3.	Calculate the proportional and integral gains by equating coefficients of the characteristic equation (pole placement).	☐
4.	Explain how the choice of design target affects the gains, and why a target slower than the plant itself leads to a negative proportional gain.	☐
5.	Use MATLAB to design and simulate a pole-placement PI controller for an identified DC motor model, and compare the result with the predicted response.	☐

2.0 Introduction

The Manual Tuning of a PID Controller Learning Guide adjusted the gains of a controller by trial and error, guided by the measured response. Manual tuning works, but it can take many experiments, it depends on the experience of the person tuning, and it gives little insight into why a particular set of gains works. Once a model of the system has been identified, for example using the Linear Least Squares for Parameter Estimation Learning Guide, a more systematic approach is possible. An important question therefore arises:

How can the gains of a PI controller be calculated directly from a model, so that the closed loop responds as required?

One answer is pole placement. The required response is first written as a damping ratio, ζ, and a natural frequency, ω_n, as in the Response Analysis of Second-Order Transfer Functions Learning Guide. These define where the poles of the closed-loop system should be. The controller gains are then calculated so that the closed-loop poles lie exactly at these locations. Every step, from the measured data to the gains, is a calculation that can be checked, repeated and explained.

Pole placement is also the design method used inside the self-tuning PI controller, where the gains are recalculated automatically every time the model changes. This is the subject of the Adaptive Control: the Self-Tuning PI Controller Learning Guide.

Why is Pole Placement Important?

Pole placement is important in control engineering because:

i. It links the performance requirements, through ζ and ω_n, directly to the controller gains, giving a traceable design.

ii. It replaces trial and error with a calculation, so a new controller can be designed as soon as a new model is available.

iii. It shows clearly how the gains depend on the system: if the motor gain doubles, the controller gains halve.

iv. The calculation is simple enough to run in real time on a microcontroller, which makes it the basis of many adaptive controllers.

3.0 The Model and the Controller

The Discrete-Time Motor Model

A DC motor driven through a zero-order hold (ZOH) and identified as a first-order model with gain K (RPM/V) and time constant τ (s) has the discrete-time transfer function:

$$G(z) = \frac{Y(z)}{U(z)} = \frac{K(1-p)}{z-p}, \qquad p = e^{-T_s/\tau} \tag{1}$$

where T_s is the sampling interval of the controller and p is the discrete-time pole of the motor. For convenience, the numerator is written as $b_s = K(1-p)$. If the model was identified at a different interval, $T_e = DT_s$, its pole $p_e = -a_1$ is converted to the control interval using $p = p_e^{1/D}$, since $e^{-Ts/\tau} = (e^{-Te/\tau})^{1/D}$.

The Discrete-Time PI Controller

As in the Discrete-Time PID Control Learning Guide, the PI controller is implemented in velocity form:

$$u_k = u_{k-1} + (K_p + K_i T_s)\, e_k - K_p e_{k-1} \tag{2}$$

where $e_k = r_k - y_k$ is the error between the reference and the measured speed. Taking the z-transform of Equation (2) gives the transfer function of the controller:

$$C(z) = \frac{U(z)}{E(z)} = \frac{q_0 z + q_1}{z-1}, \qquad q_0 = K_p + K_i T_s, \qquad q_1 = -K_p \tag{3}$$

The pole at $z = 1$ is the integrator, which removes the steady-state error, and the zero at $z = -q_1/q_0$ is set by the ratio of the two gains. The velocity form is convenient in practice because the output changes smoothly when the gains change, and anti-windup is achieved simply by clamping u_k to the actuator limits. The closed-loop system is shown in Figure 1.

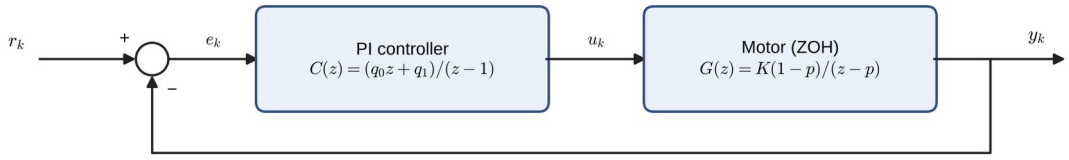


Figure 1: *Closed-Loop Speed Control with a Discrete-Time PI Controller and a First-Order Motor Model*

4.0 Specifying the Closed-Loop Poles

From Requirements to Poles

The closed-loop system of Figure 1 is second order. Its required behaviour is specified by a damping ratio, ζ, and a natural frequency, ω_n, which are chosen from the requirements for overshoot and rise time (Response Analysis of Second-Order Transfer Functions Learning Guide). The corresponding continuous-time poles are $s = -\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$. As in The z-Plane: Stability, Performance and the Jury Test Learning Guide, these are mapped to the z-plane using $z = e^{sTs}$, which gives the desired characteristic polynomial:

$$z^2 + p_1 z + p_2 = 0 \tag{4}$$

$$p_1 = -2e^{-\zeta\omega_n T_s}\cos\left(\omega_n T_s\sqrt{1-\zeta^2}\right), \qquad p_2 = e^{-2\zeta\omega_n T_s} \tag{5}$$

For $\zeta > 1$ the cosine becomes a hyperbolic cosine, $\cosh\left(\omega_n T_s\sqrt{\zeta^2-1}\right)$, and for critical damping, $\zeta = 1$, the expression simplifies to $p_1 = -2e^{-\omega nTs}$ with a double pole at $z = e^{-\omega nTs}$.

Figure 2 (left) shows the motor pole and the desired closed-loop poles for two natural frequencies. A faster design moves the poles further from $z = 1$, towards the origin.

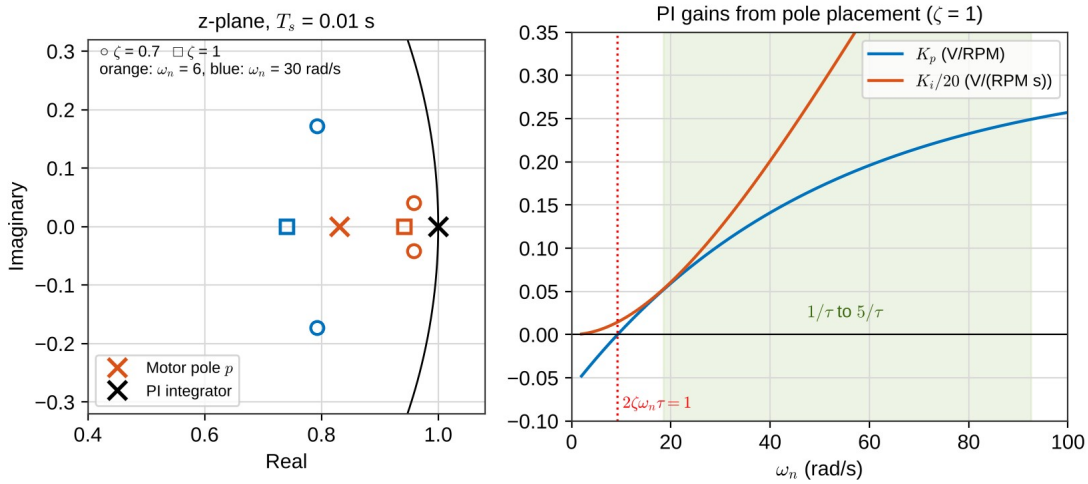


Figure 2: *Motor Pole and Desired Closed-Loop Poles on the z-Plane (Left), and the PI Gains Given by Pole Placement as the Natural Frequency Increases, for K = 16 RPM/V and τ = 0.054 s (Right)*

5.0 Pole Placement Design

Equating Coefficients

The closed-loop characteristic equation is $1 + C(z)G(z) = 0$. Substituting Equations (1) and (3):

$$(z-1)(z-p) + b_s\,(q_0 z + q_1) = 0 \tag{6}$$

Expanding:

$$z^2 + (b_s q_0 - 1 - p)\,z + (p + b_s q_1) = 0 \tag{7}$$

For the closed-loop poles to lie at the desired locations, Equation (7) must be identical to Equation (4). Equating the coefficients of z^1 and z^0 gives two equations in the two unknowns:

$$b_s q_0 - 1 - p = p_1, \qquad p + b_s q_1 = p_2 \tag{8}$$

so that:

$$q_0 = \frac{p_1 + 1 + p}{b_s}, \qquad q_1 = \frac{p_2 - p}{b_s} \tag{9}$$

and finally, using $K_p = -q_1$ and $K_i = (q_0+q_1)/T_s$:

$$K_p = \frac{p - p_2}{b_s}, \qquad K_i = \frac{1 + p_1 + p_2}{b_s T_s} \tag{10}$$

Equation (10) is the complete design. Both gains are inversely proportional to $b_s = K(1-p)$: if the gain of the motor doubles, the controller gains halve, and the closed loop is unchanged. This observation is the starting point for adaptive control.

Example 1: Designing a PI speed controller by pole placement

A DC motor has been identified as K = 16 RPM/V and τ = 0.054 s. Design a PI controller, sampled at T_s = 0.01 s, for a critically damped closed loop (ζ = 1) with ω_n = 30 rad/s.

Solution:

The motor pole and numerator are:

$$p = e^{-0.01/0.054} = 0.8310, \qquad b_s = 16(1 - 0.8310) = 2.705$$

For ζ = 1 and $\omega_n T_s$ = 0.3, Equation (5) gives:

$$p_1 = -2e^{-0.3} = -1.4816, \qquad p_2 = e^{-0.6} = 0.5488$$

Equation (10) then gives the gains:

$$K_p = \frac{0.8310 - 0.5488}{2.705} = 0.104 \text{ V/RPM}$$

$$K_i = \frac{1 - 1.4816 + 0.5488}{2.705 \times 0.01} = 2.48 \text{ V/(RPM s)}$$

Check: with these gains, Equation (7) is $z^2 - 1.4816z + 0.5488 = 0$, which has a double root at $z = 0.7408 = e^{-0.3}$, as required.

Choosing the Design Target

Equation (10) gives a positive proportional gain only if $p > p_2$. For ζ = 1:

$$K_p > 0 \quad \Leftrightarrow \quad e^{-T_s/\tau} > e^{-2\zeta\omega_n T_s} \quad \Leftrightarrow \quad 2\zeta\omega_n\tau > 1 \tag{11}$$

If the design asks for a closed loop that is slower than the motor itself, $2\zeta\omega_n\tau < 1$, the calculation gives a negative proportional gain. This is mathematically correct, since the controller must slow the motor down to achieve the requested poles, but a negative proportional gain is not a sensible controller, and in practice it is clipped to zero. The controller then becomes integral-only, and its poles are no longer where they were placed. This is shown in Figure 2 (right): K_p crosses zero at $\omega_n = 1/(2\zeta\tau)$, about 9.3 rad/s for this motor.

Example 2: A design target that is too slow

Repeat Example 1 with ω_n = 6 rad/s, the value used in the ACE-Lab teaching activity on adaptive control.

Solution:

Here $2\zeta\omega_n\tau = 2(1)(6)(0.054) = 0.65 < 1$, so a negative proportional gain is expected. With $\omega_n T_s$ = 0.06:

$$p_1 = -2e^{-0.06} = -1.8835, \qquad p_2 = e^{-0.12} = 0.8869$$

$$K_p = \frac{0.8310 - 0.8869}{2.705} = -0.021 \text{ V/RPM} \;\rightarrow\; 0$$

$$K_i = \frac{1 - 1.8835 + 0.8869}{2.705 \times 0.01} = 0.125 \text{ V/(RPM s)}$$

The proportional gain is clipped to zero and the controller is integral-only. The design is not wrong because of the model: the model is accurate, but the specification asked for a loop slower than the motor. A sensible range for this motor is $1/\tau \le \omega_n \le 5/\tau$, that is, about 18 to 90 rad/s.

The upper end of the range is set by practical limits rather than by the mathematics. Higher values of ω_n give larger gains, so the actuator saturates on

larger steps, and measurement noise is amplified: on the ACE-Lab, one encoder count per 0.01 s sample is about 1.68 RPM, so a proportional gain of 0.2 V/RPM turns each count of noise into about 0.34 V of chatter in the motor voltage. The sampling interval also limits the design: $\omega_n T_s$ should remain well below 1, so that the desired poles are not too close to the origin.

The Effect of the Controller Zero

The design places the poles exactly, but the PI controller also adds a zero to the closed loop, at $z = -q_1/q_0 = K_p/(K_p+K_iT_s)$. The closed loop is therefore not a pure second-order system, and the standard formulas give only an approximate prediction. For a critically damped second-order system, the 10–90% rise time is approximately $3.36/\omega_n$, but the zero speeds up the response and can add a little overshoot. Table 2 compares the prediction with a simulation of the model in Example 1, for a 20 RPM step and without actuator limits.

Table 2: *Pole-Placement Designs for K = 16 RPM/V, τ = 0.054 s and ζ = 1 (Simulated 20 RPM Step)*

ω_n (rad/s)	K_p (V/RPM)	K_i (V/(RPM s))	Predicted rise time, $3.36/\omega_n$ (s)	Simulated rise time (s)	Simulated overshoot (%)
6	0 (clipped)	0.125	0.56	0.97	0
18.5 (1/τ)	0.052	1.06	0.18	0.12	0
30	0.104	2.48	0.11	0.04	1.3
60	0.196	7.53	0.06	0.01	9.6

With ω_n = 6 rad/s the clipped design is much slower than predicted, because the poles are no longer where they were placed. For the other designs the response is faster than the second-order prediction because of the zero. In every case the simulated response, rather than the formula, should be checked against the requirements before the controller is used on the hardware.

6.0 Example: Pole-Placement PI Design for a DC Motor (Simulation)

In this example, the identified ACE-Lab motor model (K = 16 RPM/V, τ = 0.054 s) is used to design four PI controllers by pole placement, with ζ = 1 and different natural frequencies. The closed-loop step responses are then simulated and compared.

MATLAB Script: PI control design by pole placement for a first-order DC motor model

```
clear; clc; close all;

%% Identified motor model and design requirements
K = 16;  tau = 0.054;               % gain [RPM/V], time constant [s]
Ts = 0.01;                          % controller sampling interval [s]
zeta = 1;                           % damping ratio
wn_list = [6, 1/tau, 30, 60];       % natural frequencies [rad/s]

p  = exp(-Ts/tau);                  % discrete-time motor pole
bs = K*(1 - p);                     % numerator of G(z)
G  = tf(bs, [1 -p], Ts);            % motor model, G(z)

figure; hold on; grid on;
for wn = wn_list
```

```
    % desired closed-loop poles: z^2 + p1*z + p2 = 0
    p1 = -2*exp(-zeta*wn*Ts)*real(cosh(wn*Ts*sqrt(zeta^2 - 1)));
    p2 = exp(-2*zeta*wn*Ts);
    % pole placement (Equation 10), Kp clipped at zero
    Kp = max((p - p2)/bs, 0);
    Ki = max((1 + p1 + p2)/(bs*Ts), 0);
    C  = tf([Kp + Ki*Ts, -Kp], [1 -1], Ts);   % PI controller, C(z)
    T  = feedback(C*G, 1);                    % closed loop
    S  = stepinfo(20*T, 'RiseTimeLimits', [0.1 0.9]);
    fprintf('wn = %5.1f: Kp = %.4f, Ki = %.3f, rise %.3f s, OS %.1f %%\n', ...
            wn, Kp, Ki, S.RiseTime, S.Overshoot);
    step(20*T, 1.5);
end
xlabel('Time'); ylabel('Speed change (RPM)');
legend(compose('\\omega_n = %.1f rad/s', wn_list), 'Location', 'southeast');
```

Results: the step responses are shown in Figure 3, and the gains, rise times and overshoots printed by the script should agree with Table 2 (`stepinfo` interpolates between samples, so the rise times may differ slightly). The design with ω_n = 6 rad/s is integral-only and takes about one second to rise. Designs between $1/\tau$ and $5/\tau$ rise in about 0.1 s or less, with little or no overshoot, while ω_n = 60 rad/s begins to overshoot.

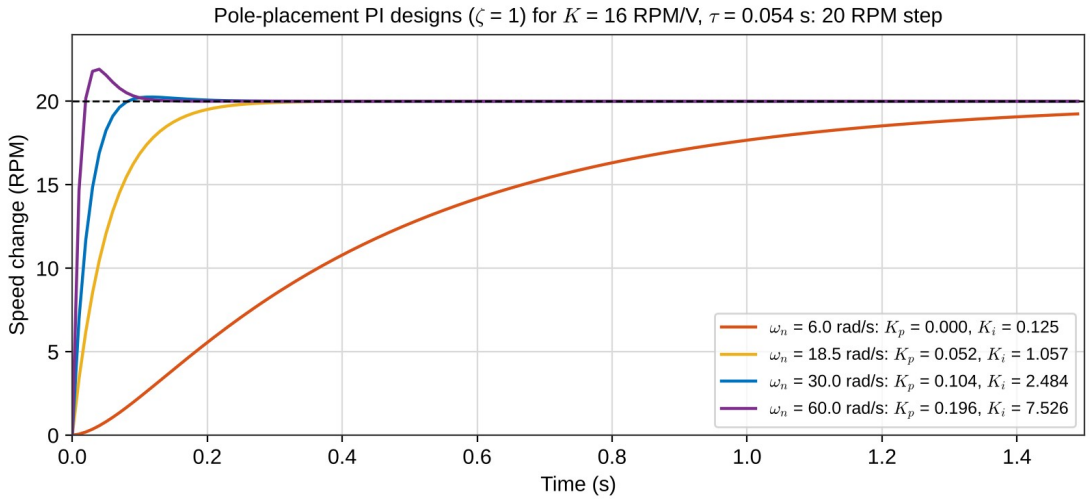


Figure 3: *Simulated 20 RPM Step Responses of Four Pole-Placement PI Designs for the Identified DC Motor Model*

Further Investigation

Investigate how the design target and errors in the model affect the closed loop:

- **Damping ratio:** repeat the design with ζ = 0.7 and ζ = 1.5 for ω_n = 30 rad/s. Record the gains, rise time and overshoot, and explain the differences.
- **Model error:** design the controller for K = 16 RPM/V, but simulate it with motors of gain 9.6 RPM/V (a load added) and 32 RPM/V (a load removed). Calculate the closed-loop poles in each case using `pole(T)` and explain why a fixed controller cannot keep the poles at the designed locations.
- **Actuator limits:** simulate the closed loop in Simulink with a saturation block that limits u_k to 0–6 V, and apply a 50 RPM step. For which values of ω_n does the response depart from Figure 3?

7.0 Summary

This Learning Guide showed how the gains of a discrete-time PI controller can be calculated directly from a first-order model by pole placement. The required response is expressed as a damping ratio and natural frequency, which define the desired closed-loop characteristic polynomial $z^2 + p_1 z + p_2$. Equating its coefficients with those of the actual characteristic equation gives the gains:

$$K_p = \frac{p - p_2}{b_s}, \qquad K_i = \frac{1 + p_1 + p_2}{b_s T_s}$$

Both gains are inversely proportional to the gain of the motor. The design target must be realistic: if the closed loop is asked to be slower than the motor ($2\zeta\omega_n\tau < 1$) the proportional gain becomes negative and is clipped to zero, while very fast targets amplify noise and saturate the actuator. A range of $1/\tau \le \omega_n \le 5/\tau$ is a good starting point. Because the PI zero also affects the response, the design should always be checked by simulation. The key progression introduced in this guide can be summarised as:

identified model (K, τ) + requirements (ζ, ω_n) → desired poles → K_p, K_i → simulated check → hardware

The design is put into practice on the ACE-Lab in the practical exercise, DC Motor PI Control Design by Pole Placement from an LLS Model, and is recalculated automatically at every sample in the Adaptive Control: the Self-Tuning PI Controller Learning Guide.

References

Åström, K. J. and Wittenmark, B. (1997). *Computer-Controlled Systems: Theory and Design*, 3rd edn. Upper Saddle River, NJ: Prentice Hall.

Franklin, G. F., Powell, J. D. and Workman, M. L. (1998). *Digital Control of Dynamic Systems*, 3rd edn. Menlo Park, CA: Addison-Wesley.

Pickering, J. E. (2026). Linear Least Squares for Parameter Estimation. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 2.0. Zenodo. https://doi.org/10.5281/zenodo.23568450

Recommended Citation

If you have used this learning guide in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). PI Control Design by Pole Placement. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671543